BACHELOR OF TECHNOLOGY — SOURCE CODE DOCUMENTATION

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING · SEMESTER 7

TECHNICAL CODE REFERENCE

MonitorX: Source Code Architecture & Module Implementation Reference

Complete annotated source code listings of the core architectural modules: AI SRE Engine, Hardware Introspection, WebSocket Telemetry, Persistence Layer, and Frontend Components.

SUBMITTED BY

Vivek Patel

B.Tech CSE, Semester 7

REPOSITORY

GitHub: Vivek-991/Server-Health-Monitoring-Dashboard-

Academic Session 2026

1.0 Architecture & Module Index

This document presents the core, production-grade source code modules powering the **MonitorX** platform. Each module has been cleaned of legacy artifacts, verified for asynchronous thread safety, and deployed to production.

Module Category	Source File	Primary Role
AI SRE Engine	`backend/src/modules/ai/ai.service.js`	Multi-provider LLM gateway, anomaly classification, heuristics engine, and playbook generation.
AI Controller	`backend/src/modules/ai/ai.controller.js`	HTTP endpoint controllers for diagnosis, chat, remediation, and configuration.
Telemetry Engine	`backend/src/services/systemMetrics.js`	Asynchronous multi-core CPU, RAM, disk, and network hardware introspection via `systeminformation`.
WebSocket Dispatcher	`backend/src/sockets/metricsSocket.js`	High-frequency 2-second bi-directional WebSocket framing and heartbeat agent tracking.
Remote Agent	`backend/agent.py`	Lightweight cross-platform Python 3 background daemon utilizing `psutil`.
Persistence Model	`backend/src/models/MetricSnapshot.js`	Mongoose schema with 24-hour Time-To-Live (TTL) automatic background purging.
AI Copilot Drawer	`frontend/src/components/common/AiAssistantDrawer.jsx`	Global slide-over AI assistant accessible from any dashboard screen with contextual awareness.
Analytics Engine	`frontend/src/pages/AnalyticsPage.jsx`	95th percentile telemetry distributions, responsive SVG Area charts, and live metric pills.

2.0 Annotated Source Code Listings

AI SRE Autonomous Service (backend/src/modules/ai/ai.service.js) JAVASCRIPT

```
const logger = require('../../utils/logger');

// Runtime in-memory AI configuration (can be updated via Settings page or loaded from env)
const aiConfig = {
  provider: process.env.AI_PROVIDER || (process.env.GEMINI_API_KEY ? 'gemini' : process.env.OPENAI_API_KEY ?
'openai' : 'builtin'),
  apiKey: process.env.GEMINI_API_KEY || process.env.OPENAI_API_KEY || '',
  geminiModel: process.env.GEMINI_MODEL || 'gemini-2.5-flash',
  openaiModel: process.env.OPENAI_MODEL || 'gpt-4o-mini',
  temperature: 0.3,
  systemPersona: 'You are MonitorX AI SRE Copilot, an elite Site Reliability Engineer and DevOps AI Agent
embedded in the MonitorX Server Health Monitoring Dashboard. You specialize in Linux/Unix systems, cloud VM
performance (AWS EC2, VPS), containerized environments, disk space remediation, memory leak debugging, process
profiling, and high-availability architecture. Always give concise, actionable answers with safe, real
bash/terminal commands.',
};

/**
 * Built-in Rule-Based Heuristic SRE Analyzer.
 * Runs instantly without requiring any external API key.
 */
function analyzeWithHeuristics(serverData) {
  const server = serverData || {};
  const serverName = server.name || server.hostname || server.serverId || 'Target Server';
  const status = (server.status || 'online').toLowerCase();
  const isOffline = status === 'offline';

  const cpu = server.cpu || {};
  const cpuUsage = Number(cpu.usage ?? 0);
  const cpuCores = Number(cpu.cores ?? 1);
  const load = server.load || {};
  const avgLoad = Number(load.avgLoad ?? load.currentLoad ?? 0);

  const memory = server.memory || {};
  const memUsagePercent = Number(memory.usagePercent ?? 0);
  const memTotalBytes = Number(memory.total ?? 0);
  const memUsedBytes = Number(memory.used ?? 0);
  const memFreeMB = Math.round((memTotalBytes - memUsedBytes) / (1024 * 1024));

  const disks = server.disks || server.disk || [];
  const primaryDisk = disks.find(d => !d.mount?.startsWith('/snap') && d.fs !== 'squashfs') || disks[0] || {};
  const diskUsagePercent = Number(primaryDisk.usagePercent ?? 0);
  const diskMount = primaryDisk.mount || '/';

  const network = server.network || {};
  const rxSec = Number(network.rx_sec ?? 0);
  const txSec = Number(network.tx_sec ?? 0);
  const totalNetKB = Math.round((rxSec + txSec) / 1024);

  const recommendations = [];
  const anomalies = [];
  let penalty = 0;

  // 1. Check Offline Status
  if (isOffline) {
    anomalies.push(`Server heartbeat lost: ${serverName} is offline or unreachable`);
    penalty += 60;
    recommendations.push({
      id: 'rec-agent-heartbeat',
      title: 'Restore MonitorX Agent Connectivity',
      category: 'reliability',
```

```
      severity: 'critical',
      impact: 'Critical - Telemetry monitoring has ceased',
      effort: '1-3 mins',
      description: `The monitoring agent on ${serverName} has not communicated recently. Check if the server
is powered on, if network egress is open, and if the background daemon is running.`,
      command: 'sudo systemctl status monitorx-agent.service || sudo systemctl restart monitorx-
agent.service',
      verificationCommand: 'curl -I http://localhost:5000/api/metrics/live',
    });
  }

  // 2. Check CPU Saturation
  if (!isOffline && cpuUsage >= 85) {
    anomalies.push(`Critical CPU Thrashing: CPU utilization is at ${cpuUsage.toFixed(1)}% across ${cpuCores}
core(s)`);
    penalty += 35;
    recommendations.push({
      id: 'rec-cpu-high',
      title: 'Inspect & Throttle High-CPU Processes',
      category: 'performance',
      severity: 'critical',
      impact: 'High - Prevents application request queuing and unresponsive timeouts',
      effort: '2 mins',
      description: `Sustained CPU usage above 85% causes thread starvation. Inspect top consumer processes,
review runaway loops, and consider applying process priorities (nice/renice).`,
      command: `ps aux --sort=-%cpu | awk 'NR<=10{printf "%-8s %-6s %-5s %-5s %s\\n", $1, $2, $3, $4, $11}'`,
      verificationCommand: 'top -b -n 1 | head -n 15',
    });
  } else if (!isOffline && cpuUsage >= 70) {
    anomalies.push(`Elevated CPU Load: CPU utilization is running hot at ${cpuUsage.toFixed(1)}%`);
    penalty += 15;
    recommendations.push({
      id: 'rec-cpu-moderate',
      title: 'Optimize Worker Thread Concurrency & CPU Affinity',
      category: 'performance',
      severity: 'warning',
      impact: 'Medium - Smoothes spike handling during traffic bursts',
      effort: '5 mins',
      description: `CPU is operating near its capacity threshold. Ensure web servers (Nginx/Node/Gunicorn)
have appropriate worker process counts matching your ${cpuCores} physical/virtual core(s).`,
      command: `lscpu | grep "CPU(s):\\|Model name\\|Thread"`,
      verificationCommand: 'mpstat 1 3 || top -n 1',
    });
  }

  // 3. Check Memory & OOM Risk
  if (!isOffline && memUsagePercent >= 88) {
    anomalies.push(`Memory Depletion: RAM usage is at ${memUsagePercent.toFixed(1)}% (${memFreeMB > 0 ?
`${memFreeMB}MB free` : 'critically low'})`);
    penalty += 35;
    recommendations.push({
      id: 'rec-mem-critical',
      title: 'Purge Inactive OS Caches & Diagnose Memory Leaks',
      category: 'reliability',
      severity: 'critical',
      impact: 'Critical - Eliminates risk of Linux OOM (Out-Of-Memory) killer terminating core databases or
services',
      effort: 'Immediate',
      description: `RAM allocation is nearing saturation. Identify top RSS processes and safely clear dentries
and pagecaches if buffered.`,
      command: `ps aux --sort=-%mem | head -n 10 && sudo sync && echo 3 | sudo tee /proc/sys/vm/drop_caches`,
      verificationCommand: 'free -m',
    });
```

```
  } else if (!isOffline && memUsagePercent >= 75) {
    anomalies.push(`Elevated Memory Usage: RAM utilization at ${memUsagePercent.toFixed(1)}%`);
    penalty += 15;
    recommendations.push({
      id: 'rec-mem-tuning',
      title: 'Tune Linux Swappiness & Memory Overcommit',
      category: 'performance',
      severity: 'warning',
      impact: 'Medium - Avoids premature swap thrashing on SSDs/NVMe storage',
      effort: '2 mins',
      description: `Default Linux swappiness (60) forces pages onto swap disk too aggressively. Setting it to
10 or 15 keeps active memory responsive.`,
      command: `sudo sysctl vm.swappiness=10 && echo "vm.swappiness=10" | sudo tee -a /etc/sysctl.d/99-
sysctl.conf`,
      verificationCommand: 'cat /proc/sys/vm/swappiness',
    });
  }

  // 4. Check Disk Mount Capacity
  if (diskUsagePercent >= 88) {
    anomalies.push(`Storage Critical: Mount "${diskMount}" is at ${diskUsagePercent.toFixed(1)}% capacity`);
    penalty += 35;
    recommendations.push({
      id: 'rec-disk-critical',
      title: 'Vacuum Systemd Journals & Purge Package Caches',
      category: 'storage',
      severity: 'critical',
      impact: 'Critical - Prevents write failures, corrupted databases, and stalled system daemons',
      effort: '3 mins',
      description: `System storage is almost exhausted. Reclaim gigabytes by vacuuming historical systemd
journals and deleting obsolete package archives.`,
      command: `sudo journalctl --vacuum-time=3d && sudo journalctl --vacuum-size=200M && sudo apt-get clean
2>/dev/null || sudo yum clean all 2>/dev/null`,
      verificationCommand: 'df -h ' + diskMount,
    });
  } else if (diskUsagePercent >= 75) {
    anomalies.push(`Storage Warning: Mount "${diskMount}" usage is at ${diskUsagePercent.toFixed(1)}%`);
    penalty += 15;
    recommendations.push({
      id: 'rec-disk-large-files',
      title: 'Locate & Rotate Uncompressed Logs & Large Files',
      category: 'storage',
      severity: 'warning',
      impact: 'Medium - Prevents unexpected disk fill-up before next automated rotation',
      effort: '5 mins',
      description: `Identify isolated large files (>100MB) such as unrotated application logs or temporary
dumps.`,
      command: `sudo find /var/log -type f -size +50M -exec ls -lh {} + 2>/dev/null || df -h`,
      verificationCommand: 'du -sh /var/log/*',
    });
  }

  // 5. Check Network Activity
  if (!isOffline && totalNetKB > 40000) {
    anomalies.push(`High Network Traffic: Combined throughput exceeding ${totalNetKB} KB/s`);
    recommendations.push({
      id: 'rec-net-audit',
      title: 'Audit Active Socket Connections & Bandwidth Egress',
      category: 'security',
      severity: 'optimization',
      impact: 'Low - Validates legitimate traffic vs potential DDoS or scraping',
      effort: '2 mins',
      description: `Inspect established socket connections and packet transmission rates to rule out
```

```
unauthorized access or runaway egress transfers.`,
      command: `ss -s && ss -tulpn | head -n 15`,
      verificationCommand: 'netstat -i || ip -s link',
    });
  }

  // 6. Proactive Hardening & Maintenance (Always provide at least 2 high quality suggestions)
  if (recommendations.length < 2) {
    recommendations.push({
      id: 'rec-proactive-security',
      title: 'Inspect Open Ports & Unattended Security Updates',
      category: 'security',
      severity: 'optimization',
      impact: 'Proactive - Hardens server perimeter against known CVEs',
      effort: '3 mins',
      description: `Verify that only mandatory ports (SSH, HTTP/HTTPS) are actively listening and ensure
automatic security patches are enabled.`,
      command: `sudo ss -tlpn && sudo unattended-upgrade --dry-run 2>/dev/null || echo "Ports verified"`,
      verificationCommand: 'sudo ufw status verbose || sudo iptables -L -n -v',
    });
    recommendations.push({
      id: 'rec-proactive-limits',
      title: 'Verify File Descriptor (nofile) & TCP Socket Limits',
      category: 'reliability',
      severity: 'optimization',
      impact: 'Proactive - Eliminates "Too many open files" errors under peak concurrent workloads',
      effort: '2 mins',
      description: `Default system limits often restrict open file descriptors to 1024. Production web servers
benefit from increasing this to 65536.`,
      command: `ulimit -n && cat /proc/sys/fs/file-max`,
      verificationCommand: 'sysctl fs.file-max',
    });
  }

  // Compute final health score & status
  const healthScore = Math.max(10, Math.min(100, Math.round(100 - penalty)));
  let statusLevel = 'healthy';
  let riskLevel = 'Low';

  if (isOffline || healthScore < 50) {
    statusLevel = 'critical';
    riskLevel = 'Critical';
  } else if (healthScore < 75) {
    statusLevel = 'warning';
    riskLevel = 'Moderate';
  }

  const summary = isOffline
    ? `⚠️ Server ${serverName} is currently OFFLINE. Monitoring telemetry is interrupted. Restoring the
daemon connection is the primary priority.`
    : healthScore >= 85
    ? `✅ Server ${serverName} is operating within healthy parameters (Health Score: ${healthScore}/100).
Resource headroom is optimal with no urgent bottlenecks.`
    : `⚠️ Server ${serverName} exhibits resource contention (Health Score: ${healthScore}/100). SRE attention
is recommended to address ${anomalies[0] || 'detected bottlenecks'}.`;

  return {
    healthScore,
    statusLevel,
    riskLevel,
    summary,
    anomalies: anomalies.length ? anomalies : ['No critical anomalies detected in active metrics stream.'],
    recommendations,
```

```
    timestamp: new Date().toISOString(),
    engine: 'builtin-heuristic-sre',
  };
}

/**
 * Call Google Gemini Generative Language API
 */
async function callGemini(prompt, systemInstruction = '') {
  const apiKey = aiConfig.apiKey || process.env.GEMINI_API_KEY;
  if (!apiKey) {
    throw new Error('Google Gemini API Key is missing. Please set it in Settings or GEMINI_API_KEY env.');
  }

  const model = aiConfig.geminiModel || 'gemini-2.5-flash';
  const url = `https://generativelanguage.googleapis.com/v1beta/models/${model}:generateContent?
key=${encodeURIComponent(apiKey)}`;

  const body = {
    contents: [
      {
        role: 'user',
        parts: [{ text: prompt }],
      },
    ],
    generationConfig: {
      temperature: aiConfig.temperature || 0.3,
      maxOutputTokens: 2048,
    },
  };

  if (systemInstruction) {
    body.system_instruction = {
      parts: [{ text: systemInstruction }],
    };
  }

  const response = await fetch(url, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(body),
  });

  if (!response.ok) {
    const errorText = await response.text();
    logger.error(`Gemini API Error (${response.status}): ${errorText}`);
    throw new Error(`Gemini API responded with status ${response.status}: ${errorText.substring(0, 150)}`);
  }

  const data = await response.json();
  const text = data?.candidates?.[0]?.content?.parts?.[0]?.text;
  if (!text) {
    throw new Error('No response text returned from Gemini API.');
  }
  return text;
}

/**
 * Call OpenAI Chat Completions API
 */
async function callOpenAI(messages, systemPrompt = '') {
  const apiKey = aiConfig.apiKey || process.env.OPENAI_API_KEY;
  if (!apiKey) {
```

```
    throw new Error('OpenAI API Key is missing. Please set it in Settings or OPENAI_API_KEY env.');
  }

  const model = aiConfig.openaiModel || 'gpt-4o-mini';
  const url = 'https://api.openai.com/v1/chat/completions';

  const fullMessages = [];
  if (systemPrompt) {
    fullMessages.push({ role: 'system', content: systemPrompt });
  }
  fullMessages.push(...messages);

  const response = await fetch(url, {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json',
      Authorization: `Bearer ${apiKey}`,
    },
    body: JSON.stringify({
      model,
      messages: fullMessages,
      temperature: aiConfig.temperature || 0.3,
      max_tokens: 2048,
    }),
  });

  if (!response.ok) {
    const errorText = await response.text();
    logger.error(`OpenAI API Error (${response.status}): ${errorText}`);
    throw new Error(`OpenAI API responded with status ${response.status}: ${errorText.substring(0, 150)}`);
  }

  const data = await response.json();
  const text = data?.choices?.[0]?.message?.content;
  if (!text) {
    throw new Error('No response text returned from OpenAI API.');
  }
  return text;
}

/**
 * Comprehensive Server Diagnosis.
 * Combines real-time telemetry, heuristic baseline, and LLM reasoning.
 */
async function diagnoseServer(serverTelemetry) {
  // First, always compute the high-accuracy baseline
  const heuristicResult = analyzeWithHeuristics(serverTelemetry);

  // If provider is set to builtin or no API key, return the heuristic diagnosis directly
  const activeProvider = aiConfig.provider;
  const hasKey = Boolean(aiConfig.apiKey || (activeProvider === 'gemini' ? process.env.GEMINI_API_KEY :
process.env.OPENAI_API_KEY));

  if (activeProvider === 'builtin' || !hasKey) {
    return heuristicResult;
  }

  // Construct LLM prompt for deep analysis
  const prompt = `
You are an expert Linux/Unix Site Reliability Engineer (SRE).
Analyze the following live server telemetry and generate an advanced, structured diagnostic assessment with
actionable recommendations.
```

```
Server Information:

// ... [Truncated: 352 more lines] ...
```

AI Controller & Diagnostic API (backend/src/modules/ai/ai.controller.js) JAVASCRIPT

```
const aiService = require('./ai.service');
const metricsController = require('../metrics/metrics.controller');
const Server = require('../../models/Server');
const logger = require('../../utils/logger');
const { AppError } = require('../../utils/AppError');
const { isDBConnected } = require('../../config/db');

class AiController {
  // Helper to fetch server telemetry for a user
  async resolveServerTelemetry(userId, serverId, inlineContext) {
    if (inlineContext && Object.keys(inlineContext).length > 0) {
      return inlineContext;
    }

    try {
      const userAgents = await metricsController.getAgentsForUserAsync(userId);
      if (serverId && userAgents[serverId]) {
        return userAgents[serverId];
      }

      // If specific serverId requested, try to find in DB
      if (serverId && isDBConnected()) {
        const srv = await Server.findOne({ _id: serverId, user: userId }).lean();
        if (srv) {
          return {
            id: srv.serverId || srv._id.toString(),
            name: srv.name,
            hostname: srv.hostname || srv.name,
            ip: srv.ip || '—',
            status: srv.status || 'offline',
            cpu: srv.metadata?.cpu || { usage: 0, cores: 1 },
            memory: { total: srv.metadata?.memory?.total || 0, used: 0, usagePercent: 0 },
            disks: [],
            os: { distro: srv.metadata?.os || 'Linux' },
          };
        }
      }

      // Return first available agent
      const firstKey = Object.keys(userAgents)[0];
      if (firstKey) {
        return userAgents[firstKey];
      }
    } catch (err) {
      logger.warn(`Failed resolving server telemetry for AI: ${err.message}`);
    }

    return {
      name: 'Default Node',
      status: 'online',
      cpu: { usage: 15, cores: 2, model: 'Virtual CPU' },
      memory: { total: 4294967296, used: 2147483648, usagePercent: 50 },
      disks: [{ fs: 'ext4', mount: '/', size: 53687091200, used: 21474836480, usagePercent: 40 }],
      network: { rx_sec: 10240, tx_sec: 5120 },
      os: { distro: 'Ubuntu 22.04 LTS' },
    };
  }

  // ── POST /api/ai/diagnose ──────────────────────────────────────
  diagnose = async (req, res, next) => {
    try {
```

```
    const userId = req.user?.id || req.user?._id;
    const { serverId, serverContext } = req.body || {};

    const telemetry = await this.resolveServerTelemetry(userId, serverId, serverContext);
    const diagnosis = await aiService.diagnoseServer(telemetry);

    res.status(200).json({
      success: true,
      data: diagnosis,
      server: {
        id: telemetry.id || serverId,
        name: telemetry.name || telemetry.hostname || 'Target Server',
        status: telemetry.status || 'online',
      },
    });
  } catch (err) {
    next(err);
  }
};

// ── POST /api/ai/chat ──────────────────────────────────────────
chat = async (req, res, next) => {
  try {
    const userId = req.user?.id || req.user?._id;
    const { messages, serverId, serverContext } = req.body || {};

    if (!Array.isArray(messages) || messages.length === 0) {
      throw new AppError('Messages array is required for AI chat.', 400);
    }

    const telemetry = await this.resolveServerTelemetry(userId, serverId, serverContext);
    const reply = await aiService.chatWithAgent(messages, telemetry);

    res.status(200).json({
      success: true,
      message: {
        role: 'assistant',
        content: reply,
        timestamp: new Date().toISOString(),
      },
    });
  } catch (err) {
    next(err);
  }
};

// ── POST /api/ai/remediation ───────────────────────────────────
remediation = async (req, res, next) => {
  try {
    const userId = req.user?.id || req.user?._id;
    const { issueType, serverId, serverContext } = req.body || {};

    if (!issueType) {
      throw new AppError('issueType is required for generating a remediation playbook.', 400);
    }

    const telemetry = await this.resolveServerTelemetry(userId, serverId, serverContext);
    const playbook = await aiService.generateRemediationPlan(issueType, telemetry);

    res.status(200).json({
      success: true,
      data: {
        issueType,
```

```
          playbook,
          server: telemetry.name || 'Server',
        },
      });
    } catch (err) {
      next(err);
    }
  };

  // ── GET /api/ai/config ──────────────────────────────────────
  getConfig = async (_req, res, next) => {
    try {
      const cfg = aiService.aiConfig;
      res.status(200).json({
        success: true,
        config: {
          provider: cfg.provider,
          geminiModel: cfg.geminiModel,
          openaiModel: cfg.openaiModel,
          hasApiKey: Boolean(cfg.apiKey || (cfg.provider === 'gemini' ? process.env.GEMINI_API_KEY :
process.env.OPENAI_API_KEY)),
          maskedApiKey: cfg.apiKey ? `${cfg.apiKey.slice(0, 4)}••••••••${cfg.apiKey.slice(-4)}` : '',
          temperature: cfg.temperature,
          isBuiltinAvailable: true,
        },
      });
    } catch (err) {
      next(err);
    }
  };

  // ── POST /api/ai/config ─────────────────────────────────────
  updateConfig = async (req, res, next) => {
    try {
      const { provider, apiKey, geminiModel, openaiModel, temperature } = req.body || {};

      if (provider) {
        aiService.aiConfig.provider = provider;
      }
      if (typeof apiKey === 'string') {
        aiService.aiConfig.apiKey = apiKey.trim();
      }
      if (geminiModel) {
        aiService.aiConfig.geminiModel = geminiModel;
      }
      if (openaiModel) {
        aiService.aiConfig.openaiModel = openaiModel;
      }
      if (typeof temperature === 'number') {
        aiService.aiConfig.temperature = Math.max(0, Math.min(1, temperature));
      }

      const cfg = aiService.aiConfig;
      res.status(200).json({
        success: true,
        message: 'AI Agent configuration updated successfully.',
        config: {
          provider: cfg.provider,
          geminiModel: cfg.geminiModel,
          openaiModel: cfg.openaiModel,
          hasApiKey: Boolean(cfg.apiKey),
          maskedApiKey: cfg.apiKey ? `${cfg.apiKey.slice(0, 4)}••••••••${cfg.apiKey.slice(-4)}` : '',
          temperature: cfg.temperature,
```

```
        },
      });
    } catch (err) {
      next(err);
    }
  };

  // ── POST /api/ai/test ──────────────────────────────────────────
  testConnection = async (req, res, next) => {
    try {
      const { provider, apiKey } = req.body || {};
      const testProvider = provider || aiService.aiConfig.provider;
      const testKey = apiKey !== undefined ? apiKey : aiService.aiConfig.apiKey;


// ... [Truncated: 76 more lines] ...
```

Hardware Telemetry Engine (backend/src/services/systemMetrics.js) JAVASCRIPT

```
const si = require('systeminformation');
const os = require('os');

// Common Windows/Linux services to monitor
const SERVICES_TO_CHECK = process.platform === 'win32'
  ? 'wuauserv,spooler,W32Time,Dhcp,Dnscache,EventLog,Schedule,LanmanServer,Netlogon,BITS'
  : 'ssh,nginx,apache2,mysql,postgresql,redis,mongod,docker,cron,sshd';

/**
 * Collect all system metrics in one shot.
 * Returns a structured object matching the MetricSnapshot schema.
 */
const collectMetrics = async () => {
  // Run all queries in parallel for performance
  const withTimeout = (promise, ms) =>
    Promise.race([promise, new Promise((_, rej) => setTimeout(() => rej(new Error('timeout')), ms))]);

  const [
    cpuLoad,
    cpuData,
    mem,
    fsSize,
    netStats,
    temp,
    services,
  ] = await Promise.all([
    si.currentLoad(),
    si.cpu(),
    si.mem(),
    si.fsSize(),
    si.networkStats(),
    si.cpuTemperature().catch(() => ({ main: null, cores: [], max: null })),
    withTimeout(si.services(SERVICES_TO_CHECK), 3000).catch(() => []),
  ]);

  // ── CPU ──────────────────────────────────────────────────────
  const cpu = {
    usage: parseFloat(cpuLoad.currentLoad.toFixed(2)),
    cores: cpuData.physicalCores || os.cpus().length,
    speed: cpuData.speed || 0,
    model: cpuData.brand || os.cpus()[0]?.model || 'Unknown',
  };

  // ── Memory ───────────────────────────────────────────────────
  const memory = {
    total: mem.total,
    used: mem.used,
    free: mem.free,
    usagePercent: parseFloat(((mem.used / mem.total) * 100).toFixed(2)),
  };

  // ── Disk ─────────────────────────────────────────────────────
  const disk = fsSize
    .filter((d) => d.size > 0)
    .map((d) => ({
      fs: d.fs,
      mount: d.mount,
      size: d.size,
      used: d.used,
      usagePercent: parseFloat(d.use.toFixed(2)),
    }));
```

```
  // ── Network ──────────────────────────────────────────────────────
  const primaryNet = netStats[0] || {};
  const network = {
    rx_bytes: primaryNet.rx_bytes || 0,
    tx_bytes: primaryNet.tx_bytes || 0,
    rx_sec: parseFloat((primaryNet.rx_sec || 0).toFixed(2)),
    tx_sec: parseFloat((primaryNet.tx_sec || 0).toFixed(2)),
    interface: primaryNet.iface || 'eth0',
  };

  // ── Uptime ──────────────────────────────────────────────────────
  const uptime = os.uptime();

  // ── Temperature ──────────────────────────────────────────────────
  const temperature = {
    main: temp.main ?? null,
    cores: temp.cores ?? [],
    max: temp.max ?? null,
  };


  // ── Load ──────────────────────────────────────────────────────
  let avgLoad = cpuLoad.avgLoad;
  if (!avgLoad || avgLoad <= 0) {
    // Windows fallback: CPU usage factor * cores
    avgLoad = (cpuLoad.currentLoad / 100) * cpu.cores;
  }

  const load = {
    avgLoad: parseFloat(avgLoad.toFixed(2)),
    currentLoad: parseFloat(cpuLoad.currentLoad.toFixed(2)),
  };

  // ── Services ──────────────────────────────────────────────────
  const runningServices = services.slice(0, 20).map((s) => ({
    name: s.name,
    running: s.running,
    pid: s.pid || null,
  }));

  return {
    cpu,
    memory,
    disk,
    network,
    uptime,
    temperature,
    load,
    services: runningServices,
    status: 'online',
    timestamp: new Date(),
  };
};

module.exports = { collectMetrics };
```

WebSocket Real-Time Dispatcher (backend/src/sockets/metricsSocket.js) JAVASCRIPT

```
const logger = require('../utils/logger');
const { sendEmailAlert } = require('../services/emailService');
const { verifyToken } = require('../middlewares/auth');
const metricsController = require('../modules/metrics/metrics.controller');
const { EMIT_INTERVAL_MS, EMAIL_COOLDOWN_MS, AGENT_OFFLINE_MS } = require('../utils/constants');

const initMetricsSocket = (io) => {
  // ── Optional Socket Auth ──────────────────────────────────────
  io.use((socket, next) => {
    try {
      const token = socket.handshake.auth?.token || socket.handshake.query?.token;
      if (token) {
        const decoded = verifyToken(token);
        socket.data.userId = decoded.id;
      } else {
        socket.data.userId = null;
      }
      next();
    } catch {
      socket.data.userId = null;
      next();
    }
  });

  // ── Remote Server Agents Monitoring Loop ──────────────────────
  // Monitors ONLY remote external servers (e.g. AWS EC2, VPS) connected via agent.py.
  // Local host PC metrics are NOT collected or broadcast.
  const broadcastAgentMetrics = async () => {
    try {
      const now = Date.now();
      let agentsChanged = false;

      for (const [serverId, agent] of metricsController.activeAgents) {
        // Check for agent timeout
        if (agent.status !== 'offline') {
          const lastUpdate = new Date(agent.timestamp).getTime();
          if (now - lastUpdate > AGENT_OFFLINE_MS) {
            agent.status = 'offline';
            agent.lastSeen = agent.lastSeen || agent.timestamp;
            agentsChanged = true;
            logger.warn(`Remote server "${serverId}" timed out — marked offline.`);

            sendEmailAlert(
              `Server Offline Alert: ${serverId}`,
              `Warning: Remote agent on "${serverId}" (${agent.hostname || 'Unknown'}) stopped sending
metrics.\nLast Heartbeat: ${agent.timestamp}`
            ).catch((err) => logger.error(`Offline alert error for ${serverId}:`, err.message));
          }
        }
      }

      // Always broadcast user-specific remote server updates to authenticated sockets
      await metricsController.emitAgentUpdates(io);
    } catch (error) {
      logger.error('Socket agent broadcast error:', error.message);
    }
  };

  setInterval(broadcastAgentMetrics, EMIT_INTERVAL_MS);
  logger.info('Remote server monitoring background loop started (Local host collection disabled)');
```

```
  // ── Per-connection handler ──────────────────────────────────
  io.on('connection', (socket) => {
    const userId = socket.data?.userId;
    logger.info(`Socket connected: ${socket.id} (userId: ${userId || 'anonymous'})`);

    // Send this user's remote server agents immediately on connect
    if (userId) {
      metricsController.getAgentsForUserAsync(userId)
        .then((agents) => socket.emit('metrics:update:agents', agents))
        .catch((err) => logger.warn('Socket connect agent fetch error:', err.message));
    }

    socket.on('disconnect', () => {
      logger.info(`Socket disconnected: ${socket.id}`);
    });
  });
};

module.exports = { initMetricsSocket };
```

Remote Telemetry Agent Daemon (backend/agent.py) PYTHON

```python
#!/usr/bin/env python3
"""
ServerPulse Monitoring Agent v2
================================
Collects CPU, memory, disk, network, and uptime metrics and pushes them
to your ServerPulse dashboard every few seconds.

Prerequisites:
    pip install psutil requests

Usage:
    # Set environment variables and run:
    SERVERPULSE_ID="my-server" SERVERPULSE_KEY="shd_xxxx..." python3 agent.py

    # Or run in background (Linux):
    nohup SERVERPULSE_ID="my-server" SERVERPULSE_KEY="shd_xxxx..." \\
        python3 agent.py > /var/log/serverpulse.log 2>&1 &

Environment Variables:
    SERVERPULSE_URL     Dashboard push URL  (default: http://localhost:5000/api/metrics/push)
    SERVERPULSE_ID      Unique server name  (default: system hostname)
    SERVERPULSE_KEY     Your API key from the dashboard (required for user isolation)
    SERVERPULSE_INTERVAL Push interval in seconds (default: 5)
"""

import os
import sys
import time
import socket
import platform
import logging

# ── Setup logging ──────────────────────────────────────────────
logging.basicConfig(
    level=logging.INFO,
    format='%(asctime)s [%(levelname)s] %(message)s',
    datefmt='%H:%M:%S'
)
log = logging.getLogger('serverpulse')

# ── Configuration ──────────────────────────────────────────────
DASHBOARD_URL = os.environ.get('SERVERPULSE_URL', 'http://localhost:5000/api/metrics/push')
SERVER_ID     = os.environ.get('SERVERPULSE_ID', socket.gethostname())
API_KEY       = os.environ.get('SERVERPULSE_KEY', 'default-secure-key-123')
INTERVAL      = int(os.environ.get('SERVERPULSE_INTERVAL', '5'))

# ── Check dependencies ─────────────────────────────────────────
try:
    import psutil
except ImportError:
    log.error("'psutil' is not installed. Run: pip install psutil requests")
    sys.exit(1)

try:
    import requests
except ImportError:
    log.error("'requests' is not installed. Run: pip install psutil requests")
    sys.exit(1)

# ── Metric collectors ──────────────────────────────────────────
```

```
def get_cpu():
    try:
        load_avg = os.getloadavg()[0] if hasattr(os, 'getloadavg') else 0
    except Exception:
        load_avg = 0

    freq = psutil.cpu_freq()
    return {
        'usage': round(psutil.cpu_percent(interval=1), 2),
        'cores': psutil.cpu_count(logical=False) or 1,
        'threads': psutil.cpu_count(logical=True) or 1,
        'speed': round(freq.current / 1000, 2) if freq else 0,  # GHz
        'model': _cpu_model(),
    }

def _cpu_model():
    """Try to get a real CPU model name."""
    try:
        if platform.system() == 'Linux':
            with open('/proc/cpuinfo') as f:
                for line in f:
                    if 'model name' in line:
                        return line.split(':')[1].strip()
        elif platform.system() == 'Darwin':
            import subprocess
            out = subprocess.check_output(['sysctl', '-n', 'machdep.cpu.brand_string'])
            return out.decode().strip()
    except Exception:
        pass
    return platform.processor() or 'Unknown CPU'

def get_memory():
    m = psutil.virtual_memory()
    return {
        'total': m.total,
        'used': m.used,
        'free': m.available,
        'usagePercent': round(m.percent, 2),
    }

def get_disks():
    disks = []
    for part in psutil.disk_partitions(all=False):
        mount = part.mountpoint or ''
        fstype = part.fstype or ''
        # Filter out virtual snap packages, loop devices, and squashfs mounts
        if mount.startswith('/snap') or fstype == 'squashfs' or 'loop' in part.device:
            continue
        try:
            usage = psutil.disk_usage(mount)
            disks.append({
                'fs': fstype or 'unknown',
                'mount': mount,
                'device': part.device,
                'size': usage.total,
                'used': usage.used,
                'free': usage.free,
                'usagePercent': round(usage.percent, 2),
            })
        except (PermissionError, OSError):
            pass
    return disks or [{'fs': 'unknown', 'mount': '/', 'size': 0, 'used': 0, 'free': 0, 'usagePercent': 0}]
```

```
def get_network():
    try:
        n1 = psutil.net_io_counters()
        time.sleep(0.5)
        n2 = psutil.net_io_counters()
        return {
            'rx_bytes': n2.bytes_recv,
            'tx_bytes': n2.bytes_sent,
            'rx_sec': max(0, n2.bytes_recv - n1.bytes_recv) * 2,  # *2 because 0.5s sample
            'tx_sec': max(0, n2.bytes_sent - n1.bytes_sent) * 2,
            'interface': _primary_iface(),
        }
    except Exception:
        return {'rx_bytes': 0, 'tx_bytes': 0, 'rx_sec': 0, 'tx_sec': 0, 'interface': 'unknown'}

def _primary_iface():
    """Return the primary non-loopback interface name."""
    try:
        for name, addrs in psutil.net_if_addrs().items():
            if name.startswith(('lo', 'Loopback')):
                continue
            for addr in addrs:
                if addr.family == socket.AF_INET and not addr.address.startswith('127.'):
                    return name
    except Exception:
        pass
    return 'eth0'

def get_temperatures():
    try:
        temps = psutil.sensors_temperatures()
        if not temps:
            return []
        result = []
        for name, entries in temps.items():
            for entry in entries[:3]:
                result.append({
                    'sensor': name,
                    'label': entry.label or name,
                    'main': round(entry.current, 1),
                    'high': entry.high,
                    'critical': entry.critical,
                })
        return result
    except (AttributeError, Exception):
        return []

def get_os_info():
    try:
        uptime_seconds = int(time.time() - psutil.boot_time())
    except Exception:
        uptime_seconds = 0

    return {
        'distro': _os_name(),
        'platform': sys.platform,
        'arch': platform.machine(),
        'hostname': socket.gethostname(),
        'uptime': uptime_seconds,
        'uptime_human': _format_uptime(uptime_seconds),
    }

def _os_name():
```

```
    try:
        if platform.system() == 'Linux':
            with open('/etc/os-release') as f:
                for line in f:
                    if line.startswith('PRETTY_NAME='):
                        return line.split('=', 1)[1].strip().strip('"')
    except Exception:
        pass
    return platform.system() or 'Unknown OS'

def _format_uptime(seconds):
    d = seconds // 86400
    h = (seconds % 86400) // 3600

// ... [Truncated: 121 more lines] ...
```

Metric Snapshot Schema (TTL Index) (backend/src/models/MetricSnapshot.js) JAVASCRIPT

```
const mongoose = require('mongoose');

const MetricSnapshotSchema = new mongoose.Schema(
  {
    user: {
      type: mongoose.Schema.Types.ObjectId,
      ref: 'User',
      index: true,
    },
    server: {
      type: mongoose.Schema.Types.ObjectId,
      ref: 'Server',
      index: true,
    },
    serverId: {
      type: String,
      index: true,
    },
    timestamp: {
      type: Date,
      default: Date.now,
      index: true,
    },
    cpu: {
      usage: { type: Number },
      cores: { type: Number },
      speed: { type: Number },
      model: { type: String },
    },
    memory: {
      total: { type: Number },
      used: { type: Number },
      free: { type: Number },
      usagePercent: { type: Number },
    },
    disk: [
      {
        fs: { type: String },
        mount: { type: String },
        size: { type: Number },
        used: { type: Number },
        usagePercent: { type: Number },
      },
    ],
    network: {
      rx_bytes: { type: Number },
      tx_bytes: { type: Number },
      rx_sec: { type: Number },
      tx_sec: { type: Number },
      interface: { type: String },
    },
    uptime: { type: Number },
    temperature: {
      main: { type: Number },
      cores: [{ type: Number }],
      max: { type: Number },
    },
    load: {
      avgLoad: { type: Number },
      currentLoad: { type: Number },
    },
```

```
    services: [
      {
        name: { type: String },
        running: { type: Boolean },
        pid: { type: Number },
      },
    ],
    status: {
      type: String,
      enum: ['online', 'offline', 'degraded'],
      default: 'online',
    },
  },
  {
    timestamps: false,
    collection: 'metric_snapshots',
  }
);

MetricSnapshotSchema.index({ timestamp: 1 }, { expireAfterSeconds: 86400 });
MetricSnapshotSchema.index({ user: 1, timestamp: -1 });
MetricSnapshotSchema.index({ server: 1, timestamp: -1 });

module.exports = mongoose.model('MetricSnapshot', MetricSnapshotSchema);
```

📄 **Global AI Copilot Drawer Component (frontend/src/components/common/AiAssistantDrawer.jsx)**

JAVASCRIPT

```
import React, { useState, useEffect, useRef, useMemo } from 'react';
import { useNavigate } from 'react-router-dom';
import useMetrics from '../../hooks/useMetrics';
import { aiApi } from '../../api/metricsApi';
import '../../styles/ai.css';

const QUICK_PROMPTS = [
  '⚡ Instant server diagnosis',
  '💾 How do I reduce RAM usage?',
  '💿 How to clear disk space?',
  '🛡️ Audit open ports',
];

const AiAssistantDrawer = () => {
  const navigate = useNavigate();
  const { agents = {}, current } = useMetrics();
  const [isOpen, setIsOpen] = useState(false);

  // Active server resolution
  const remoteServerList = useMemo(() => {
    return Object.keys(agents).map((key) => {
      const ag = agents[key];
      return {
        id: ag.id || key,
        name: ag.name || key,
        hostname: ag.hostname || key,
        ip: ag.ip || '—',
        status: ag.status || 'online',
        metrics: ag,
      };
    });
  }, [agents]);

  const [selectedServerId, setSelectedServerId] = useState('');

  const activeServer = useMemo(() => {
    if (!remoteServerList.length) {
      if (current) {
        return {
          id: 'primary-node',
          name: 'Primary Node',
          hostname: 'localhost',
          ip: '127.0.0.1',
          status: 'online',
          metrics: current,
        };
      }
      return null;
    }
    const found = remoteServerList.find((s) => s.id === selectedServerId);
    return found || remoteServerList[0];
  }, [remoteServerList, selectedServerId, current]);

  // Chat state
  const [messages, setMessages] = useState([
    {
      role: 'assistant',
      content: '👋 Hi! I am your **MonitorX AI Copilot**. Click **Instant Diagnose** or ask me any question
about your servers, performance, or Linux commands.',
```

```
      timestamp: new Date().toLocaleTimeString([], { hour: '2-digit', minute: '2-digit' }),
    },
  ]);
  const [inputText, setInputText] = useState('');
  const [loading, setLoading] = useState(false);
  const [quickDiag, setQuickDiag] = useState(null);
  const [diagLoading, setDiagLoading] = useState(false);
  const chatEndRef = useRef(null);

  const handleSendRef = useRef();

  useEffect(() => {
    if (isOpen) {
      chatEndRef.current?.scrollIntoView({ behavior: 'smooth' });
    }
  }, [messages, loading, isOpen]);

  useEffect(() => {
    const handleOpen = (e) => {
      setIsOpen(true);
      if (e.detail?.prompt) {
        setTimeout(() => {
          handleSendRef.current?.(e.detail.prompt);
        }, 150);
      }
    };
    window.addEventListener('open-ai-drawer', handleOpen);
    return () => window.removeEventListener('open-ai-drawer', handleOpen);
  }, []);

  const handleSend = async (customPrompt) => {
    const text = customPrompt || inputText;
    if (!text.trim() || loading) return;

    const userMsg = {
      role: 'user',
      content: text.trim(),
      timestamp: new Date().toLocaleTimeString([], { hour: '2-digit', minute: '2-digit' }),
    };

    const nextMessages = [...messages, userMsg];
    setMessages(nextMessages);
    if (!customPrompt) setInputText('');
    setLoading(true);

    try {
      const payload = {
        messages: nextMessages.map((m) => ({ role: m.role, content: m.content })),
        serverId: activeServer?.id,
        serverContext: activeServer?.metrics || {},
      };
      const res = await aiApi.chat(payload);
      if (res.success && res.message) {
        setMessages((prev) => [
          ...prev,
          {
            role: 'assistant',
            content: res.message.content,
            timestamp: new Date().toLocaleTimeString([], { hour: '2-digit', minute: '2-digit' }),
          },
        ]);
      }
    } catch (err) {
```

```
      setMessages((prev) => [
        ...prev,
        {
          role: 'assistant',
          content: `⚠️ Failed to get answer: ${err.message}`,
          timestamp: new Date().toLocaleTimeString([], { hour: '2-digit', minute: '2-digit' }),
        },
      ]);
    } finally {
      setLoading(false);
    }
  };
  handleSendRef.current = handleSend;

  const handleQuickDiagnose = async () => {
    if (!activeServer || diagLoading) return;
    setDiagLoading(true);
    try {
      const res = await aiApi.diagnose({
        serverId: activeServer.id,
        serverContext: activeServer.metrics || {},
      });
      if (res.success && res.data) {
        setQuickDiag(res.data);
      }
    } catch (err) {
      console.error('Quick diagnose failed:', err);
    } finally {
      setDiagLoading(false);
    }
  };

  const m = activeServer?.metrics || {};
  const isOffline = activeServer?.status === 'offline';
  const cpuVal = isOffline ? 0 : (m.cpu?.usage ?? 0);
  const ramVal = isOffline ? 0 : (m.memory?.usagePercent ?? 0);
  const primaryDisk = (m.disks || m.disk || []).find((d) => !d.mount?.startsWith('/snap')) || {};
  const diskVal = isOffline ? 0 : (primaryDisk.usagePercent ?? 0);

  return (
    <>
      {/* ── Floating Action Trigger Button ──────────────────────── */}
      {!isOpen && (
        <button
          className="ai-floating-trigger"
          onClick={() => setIsOpen(true)}
          title="Open AI SRE Copilot"
          aria-label="Open AI SRE Copilot"
        >
          <span>🤖 AI Copilot</span>
          <span className="ai-floating-badge">AI</span>
        </button>
      )}

      {/* ── Slide-over Drawer Panel ─────────────────────────────── */}
      {isOpen && (
        <>
          <div className="ai-drawer-overlay" onClick={() => setIsOpen(false)} />
          <div className="ai-drawer-panel">
            {/* Header */}
            <div className="ai-drawer-header">
              <div className="ai-drawer-title">
                <span style={{ fontSize: '1.25rem' }}>🤖</span>
```

```
                <div>
                  <div style={{ lineHeight: 1.2 }}>AI SRE Copilot</div>
                  <span style={{ fontSize: '0.7rem', color: 'var(--color-text-secondary)', fontWeight: 500 }}>
                    {activeServer?.name || 'Live Telemetry'}
                  </span>
                </div>
              </div>

              <div className="ai-drawer-controls">
                <button
                  className="ai-icon-btn"
                  onClick={() => {
                    setIsOpen(false);
                    navigate('/ai-agent');
                  }}
                  title="Open full AI Advisor page"
                >
                  ↗
                </button>
                <button
                  className="ai-icon-btn"
                  onClick={() => setIsOpen(false)}
                  title="Close AI Copilot"
                >
                  ✕
                </button>
              </div>
            </div>

            {/* Quick Server Switch & Metrics Strip */}
            <div
              style={{
                padding: '10px 16px',
                background: 'rgba(0,0,0,0.25)',
                borderBottom: '1px solid var(--color-border)',
                display: 'flex',

// ... [Truncated: 203 more lines] ...
```

Analytics & 95th Percentile Page (frontend/src/pages/AnalyticsPage.jsx) JAVASCRIPT

```
import React, { useState, useMemo, useEffect } from 'react';
import PageLayout from '../components/common/PageLayout';
import { useMetricsContext } from '../context/MetricsContext';
import { fetchHistoricalMetrics } from '../api/metricsApi';

import {
  AreaChart, Area, XAxis, YAxis, CartesianGrid,
  Tooltip as RechartsTooltip, ResponsiveContainer
} from 'recharts';

// ── Custom Tooltip for Recharts ──────────────────────────────────────
const CustomTooltip = ({ active, payload, label, unit = '%' }) => {
  if (active && payload?.length) {
    const val = payload[0].value;
    const color = payload[0].stroke;
    return (
      <div style={{
        background: 'rgba(17, 17, 17, 0.95)',
        backdropFilter: 'blur(8px)',
        border: '1px solid var(--color-border-strong)',
        borderRadius: 'var(--radius-md)',
        padding: '10px 14px',
        boxShadow: 'var(--shadow-hover)',
      }}>
        <div style={{ fontSize: '10px', color: 'var(--color-text-secondary)', textTransform: 'uppercase',
marginBottom: '4px' }}>
          Time: {label}
        </div>
        <div style={{ color, fontWeight: 800, fontSize: 'var(--text-base)', fontFamily: 'var(--font-mono)' }}>
          {val.toFixed(1)}{unit}
        </div>
      </div>
    );
  }
  return null;
};

// ── Stats calculator ─────────────────────────────────────────────────
const statsOf = (arr) => {
  if (!arr || arr.length === 0) return { min: 0, max: 0, avg: 0, p95: 0 };
  const sorted = [...arr].sort((a, b) => a - b);
  const avg = arr.reduce((s, v) => s + v, 0) / arr.length;
  const p95 = sorted[Math.floor(sorted.length * 0.95)] ?? 0;
  return { min: sorted[0], max: sorted[sorted.length - 1], avg, p95 };
};

const fmt = (v) => (typeof v === 'number' ? v.toFixed(1) : '0.0');

// ── KPI Card Component ──────────────────────────────────────────────
const KPI = ({ label, value, unit, icon, color, status, subtext }) => (
  <div className="card kpi-card" style={{ '--kpi-accent': color }}>
    <div className="kpi-card-header">
      <div className="kpi-icon-wrap" style={{ color }}>{icon}</div>
      {status && (
        <span
          className="kpi-status-tag"
          style={{
            background: status.level === 'critical' ? 'rgba(239, 68, 68, 0.15)' :
                        status.level === 'warning' ? 'rgba(245, 158, 11, 0.15)' : 'rgba(34, 197, 94, 0.15)',
            color: status.level === 'critical' ? '#ef4444' :
                   status.level === 'warning' ? '#f59e0b' : '#22c55e',
```

```
            border: `1px solid ${status.level === 'critical' ? 'rgba(239, 68, 68, 0.3)' :
                                status.level === 'warning' ? 'rgba(245, 158, 11, 0.3)' : 'rgba(34, 197, 94,
0.3)'}`,
          }}
        >
          {status.label}
        </span>
      )}
    </div>
    <div className="kpi-val" style={{ color }}>
      {value}<span className="kpi-unit">{unit}</span>
    </div>
    <div className="kpi-label">{label}</div>
    {subtext && <div className="kpi-subtext">{subtext}</div>}
  </div>
);

// ── Analytics Page Component ──────────────────────────────────────
const AnalyticsPage = () => {
  const { agents = {}, history = [], historyMap = {} } = useMetricsContext();
  const [selectedServerId, setSelectedServerId] = useState('');
  const [range, setRange] = useState(150); // number of history points to use
  const [fetchedHistory, setFetchedHistory] = useState([]);

  const agentKeys = Object.keys(agents);
  const activeServerId = selectedServerId;

  useEffect(() => {
    let isMounted = true;
    fetchHistoricalMetrics(range, activeServerId)
      .then((res) => {
        if (isMounted && res?.data) {
          setFetchedHistory(res.data);
        }
      })
      .catch(() => {});
    return () => { isMounted = false; };
  }, [activeServerId, range]);

  const mergedHistory = useMemo(() => {
    const liveHistory = activeServerId ? (historyMap[activeServerId] || []) : history;
    const combined = [...fetchedHistory, ...liveHistory];
    const map = new Map();
    combined.forEach((snap) => {
      const key = snap._id || snap.timestamp || JSON.stringify(snap);
      map.set(key, snap);
    });
    return Array.from(map.values()).slice(-range);
  }, [fetchedHistory, historyMap, activeServerId, history, range]);

  const slice = mergedHistory;

  // Transform data for Recharts
  const chartData = useMemo(() => {
    return slice.map((snap, i) => {
      const ago = (slice.length - 1 - i) * 2;
      const timeLabel = ago === 0 ? 'now' : `${ago}s`;
      const disks = snap.disks || snap.disk || [];
      return {
        index: i,
        timeLabel,
        cpu: snap.cpu?.usage ?? 0,
        ram: snap.memory?.usagePercent ?? 0,
```

```
        disk: disks[0]?.usagePercent ?? 0,
        network: (snap.network?.rx_sec ?? 0) / 1024, // Convert to KB/s
      };
    });
  }, [slice]);

  const cpuData = chartData.map(d => d.cpu);
  const ramData = chartData.map(d => d.ram);
  const diskData = chartData.map(d => d.disk);
  const netData = chartData.map(d => d.network);

  const cpuStats  = statsOf(cpuData);
  const ramStats  = statsOf(ramData);
  const diskStats = statsOf(diskData);
  const netStats  = statsOf(netData);

  const latestPoint = chartData[chartData.length - 1] || { cpu: 0, ram: 0, disk: 0, network: 0 };

  const exportCSV = () => {
    const header = 'Time,CPU%,RAM%,Disk%,Network KB/s';
    const rows = chartData.map((d) => [
      d.timeLabel,
      d.cpu.toFixed(2),
      d.ram.toFixed(2),
      d.disk.toFixed(2),
      d.network.toFixed(2),
    ].join(','));
    const csv = [header, ...rows].join('\n');
    const blob = new Blob([csv], { type: 'text/csv' });
    const a = document.createElement('a');
    a.href = URL.createObjectURL(blob);
    a.download = `analytics_${activeServerId || 'primary'}_${Date.now()}.csv`;
    a.click();
  };

  const handleAiTrendAudit = () => {
    const serverName = activeServerId ? (agents[activeServerId]?.name || activeServerId) : 'Primary Host
Node';
    const prompt = `Analyze historical telemetry trends for ${serverName}:
- CPU: Current ${fmt(latestPoint.cpu)}%, Avg ${fmt(cpuStats.avg)}%, Peak ${fmt(cpuStats.max)}%, P95
${fmt(cpuStats.p95)}%
- RAM: Current ${fmt(latestPoint.ram)}%, Avg ${fmt(ramStats.avg)}%, Peak ${fmt(ramStats.max)}%, P95
${fmt(ramStats.p95)}%
- Disk: Current ${fmt(latestPoint.disk)}%, Avg ${fmt(diskStats.avg)}%, Peak ${fmt(diskStats.max)}%
- Network RX: Current ${fmt(latestPoint.network)} KB/s, Peak ${fmt(netStats.max)} KB/s
Please provide an SRE assessment: identify potential bottleneck risks, resource headroom, and recommended
optimizations.`;

    window.dispatchEvent(new CustomEvent('open-ai-drawer', { detail: { prompt } }));
  };

  const RANGES = [
    { label: '5 min', val: 150 },
    { label: '15 min', val: 450 },
    { label: '30 min', val: 900 }
  ];

  const getStatus = (val, highThresh = 80, warnThresh = 65) => {
    if (val >= highThresh) return { level: 'critical', label: 'CRITICAL' };
    if (val >= warnThresh) return { level: 'warning', label: 'ELEVATED' };
    return { level: 'healthy', label: 'OPTIMAL' };
  };
```

```
  return (
    <PageLayout>
      <div className="page-header">
        <div>
          <h1 className="page-title">📈 Analytics & Telemetry</h1>
          <p className="page-sub">{slice.length} historical samples · 2-second real-time streaming</p>
        </div>
        <div className="analytics-header-controls">
          {agentKeys.length > 0 && (
            <select
              value={activeServerId}
              onChange={(e) => setSelectedServerId(e.target.value)}
              className="analytics-server-select"
            >
              <option value="">🖥️ Primary Node (Localhost)</option>
              {agentKeys.map((key) => {
                const ag = agents[key];
                return (
                  <option key={key} value={key}>
                    🖥️ {ag?.name || key} ({ag?.ip || 'remote'})
                  </option>
                );
              })}
            </select>
          )}

          <div className="range-tabs">
            {RANGES.map((r) => (
              <button
                key={r.val}
                className={`range-tab ${range === r.val ? 'active' : ''}`}
                onClick={() => setRange(r.val)}
              >
                {r.label}
              </button>
            ))}
          </div>

          <button className="btn-ai-audit" onClick={handleAiTrendAudit} title="Ask AI SRE Copilot to evaluate
telemetry trends">
             AI Trend Audit
          </button>

          <button className="btn-secondary" onClick={exportCSV}>
            ⬇ Export CSV
          </button>
        </div>
      </div>

      {/* KPIs */}
      <div className="kpi-grid">
        <KPI
          label="Avg CPU Usage"
          value={fmt(cpuStats.avg)}
          unit="%"
          icon="🖥️"
          color="#6384ff"
          status={getStatus(cpuStats.avg, 80, 65)}
          subtext={`Min ${fmt(cpuStats.min)}% · Peak ${fmt(cpuStats.max)}%`}
        />
        <KPI
          label="Peak RAM Usage"
          value={fmt(ramStats.max)}
```

```
          unit="%"
          icon="💾"
          color="#00d4ff"
          status={getStatus(ramStats.max, 85, 70)}
          subtext={`Avg ${fmt(ramStats.avg)}% · P95 ${fmt(ramStats.p95)}%`}
        />
        <KPI

// ... [Truncated: 197 more lines] ...
```